SPA vs sito vetrina: rendering lato server

Corso di Frontend Development – Modulo 04

Federico Giuggioloni

ITS – Corso di Frontend Development

Anno formativo 2026/2027

Contenuto del modulo

1. SPA vs sito vetrina: rendering lato server
1.1 Due problemi diversi
1.2 Modalità di rendering
1.3 SEO e prestazioni percepite
1.4 La vetrina con Next.js
1.5 Il blog con Astro
1.6 PHP e le altre strade
1.7 I due rami, e il laboratorio

Sommario del modulo

1. SPA vs sito vetrina: rendering lato server
1.1 Due problemi diversi
1.2 Modalità di rendering
1.3 SEO e prestazioni percepite
1.4 La vetrina con Next.js
1.5 Il blog con Astro
1.6 PHP e le altre strade
1.7 I due rami, e il laboratorio

App e sito non sono la stessa cosa

App (admin e carrello di minicommerce)

- Utenti **loggati**: admin e clienti
- Molta interazione, poca "lettura"
- Non deve essere trovata da Google
- Si usa per ore: il primo caricamento conta poco
- SPA: scelta giusta

Vetrina (il negozio online)

- Visitatori **anonimi**, contenuti pubblici
- Poca interazione, molta lettura
- **Deve** essere trovata: SEO, anteprime social
- Visita di 30 secondi: il primo caricamento è tutto
- SPA: scelta sbagliata

Il problema di minicommerce oggi

Il catalogo è una SPA React: HTML vuoto, tutto il JavaScript prima di vedere un prezzo. Per un negozio che vive di ricerche su Google è il difetto più costoso di tutti.

Cosa vede un crawler sul catalogo

```
<!doctype html>
<html lang="it"><head><title>minicommerce</title></head>
<body>
  <div id="root"></div>
  <script type="module" src="/assets/index-8f2c.js"></script>
</body></html>
```

- Googlebot *esegue* JavaScript, ma in una seconda fase, con ritardo e budget limitato.
- Bing, i crawler di WhatsApp/LinkedIn/Slack per le anteprime, i comparatori di prezzo: **non lo eseguono**.
- Nessun prezzo, nessuna marca, nessuna disponibilità nell'HTML: niente risultati ricchi, niente anteprima della scheda.
- La scheda articolo un URL ce l'ha (`/article/:articleid`), la categoria no: i filtri del catalogo vivono nello stato React e nella query string.
- In ogni caso titolo e descrizione restano quelli di `index.html`, uguali per tutte le rotte: `<title>minicommerce</title>` e basta.

La prova, in una riga di comando

```
node scripts/confronto-seo.mjs --id 1
```

Stessa scheda, stesso articolo	SPA (Vite)	Vetrina (Next)
byte di HTML	632	39 111
parole di testo	1	125
`<title>` specifico	−	+
`description`	−	+
`canonical`	−	+
`<h1>`	−	+
blocchi JSON-LD	0	4
link `<a href>`	0	22
nome dell'articolo	assente	presente
prezzo	assente	presente

Da leggere ad alta voce

La SPA non sta sbagliando niente: fa il suo mestiere, che è far *usare* l'applicazione. Solo che il contenuto arriva dopo, in JavaScript, e in questo HTML non c'è. La vetrina fa il mestiere opposto: farsi *leggere*.

La divisione dei ruoli che adottiamo

Vetrina pubblica (ramo `web/nextjs`)

- `/`, `/categorie/monitor`, `/chi-siamo`
- `/prodotti/1-thinkpad-x1-...`: una per articolo
- Pregenerata al build, rigenerata quando il catalogo cambia
- Indicizzabile, veloce, quasi senza JavaScript

Area applicativa (modulo 1)

- `/login`, `/cart`, `/admin`: SPA React + Vite
- Protetta, dati sempre freschi a ogni apertura
- Nessun bisogno di SEO: un carrello non si indicizza

Due applicazioni, una sola API

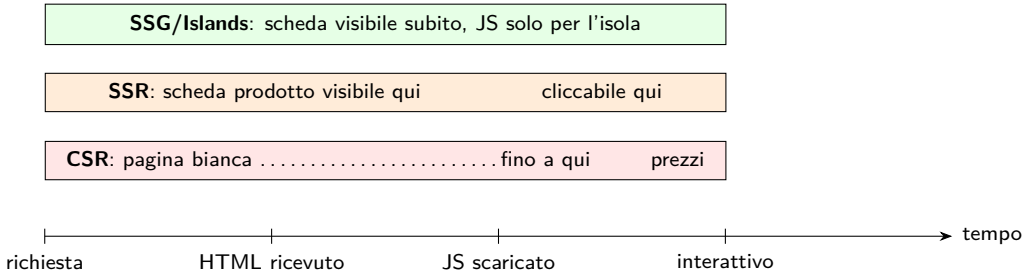
Entrambe parlano con lo stesso backend Express su `:3001`. La vetrina usa solo le rotte pubbliche in lettura (`GET /api/categories`, `GET /api/products`); l'app usa anche quelle protette: `/api/cart` e `POST /api/orders` per i clienti, le scritture sui prodotti per gli admin.

Il vocabolario

Sigla	Cosa succede	Esempio
CSR	Il browser riceve HTML vuoto e costruisce tutto con JS	la SPA di minicommerce
SSR	Il server genera l'HTML *a ogni richiesta*, poi il JS "idrata"	Next.js, PHP
SSG	L'HTML è generato *al build*, servito come file statico	Astro, Hugo
ISR	Statico, ma rigenerato dopo una scadenza o su richiesta	Next.js
Streaming	Il server invia l'HTML a pezzi man mano che è pronto	React 19, Next
PPR	Guscio statico spedito subito, i pezzi dinamici in streaming	Next 16
Islands	Pagina statica con "isole" interattive idratate a parte	Astro
RSC	Componenti React eseguiti *solo* sul server, zero JS al client	Next.js App Router

- Nessuna è "la migliore": un sito le **mescola** pagina per pagina. Nella vetrina Next ce ne sono cinque (SSR, SSG, ISR, streaming, PPR, tutte su RSC); nel blog Astro ci sono SSG e islands. La CSR è la SPA del modulo 1.

CSR vs SSR vs SSG: la linea del tempo



- **Idratazione**: React riprende in mano l'HTML già disegnato dal server e ci attacca gli eventi. Nel frattempo la pagina si vede ma non risponde.
- Meno JavaScript = idratazione più breve = pagina utilizzabile prima.

Come scegliere, pagina per pagina

Pagina	Perché	Modalità
`/chi-siamo`	non ha dati, cambia una volta all'anno	statica
`/categorie`	sette categorie, cambiano raramente	statica
`/categorie/:slug`	una per categoria, dal catalogo	SSG + ISR
`/prodotti/:slug`	una per articolo: è il contenuto che deve finire su Google	SSG + ISR
`/` con i filtri	dipende dalla query string: infinite combinazioni	dinamica
`sitemap.xml`, `robots.txt`	nascono dai dati	statiche
`/api/revalidate`	è un'azione, non una pagina	dinamica
`/login`, `/cart`, `/admin`	hanno un utente e un token	la SPA del modulo 1

Regola pratica

Statico finché possibile, server quando i dati cambiano a ogni richiesta, client solo per ciò che è davvero interattivo: è il confine fra pagina indicizzabile e area applicativa.

SEO tecnica: il minimo indispensabile

- `<title>` unico per pagina (50–60 caratteri): articolo, marca, negozio.
- `<meta name="description">` (150–160 caratteri): è il testo dello snippet.
- Un solo `<h1>`, gerarchia `h2`/`h3` sensata, HTML semantico.
- URL leggibili: `/categorie/monitor` e non `/?categoryId=3`.
- `<link rel="canonical">`: un elenco filtrato per prezzo non è una pagina nuova.
- `sitemap.xml` e `robots.txt`.
- Open Graph: titolo, descrizione, immagine 1200×630.
- Dati strutturati **JSON-LD**: per un negozio, `Product` e `BreadcrumbList`.
- Niente `noindex` dimenticato in produzione.
- Google Search Console: come Google vi vede davvero.

La testa di una scheda prodotto (quella vera)

```
<title>ThinkPad X1 Carbon Gen 12 | minicommerce</title>
<meta name="description" content="Ultrabook 14&amp;quot; con Intel Core Ultra 7,
      32GB RAM, 1TB SSD. Leggero e robusto, ideale per il lavoro in mobilità.">
<link rel="canonical"
      href="http://localhost:3000/prodotti/1-thinkpad-x1-carbon-gen-12">
<meta property="og:type" content="website">
<meta property="og:title" content="ThinkPad X1 Carbon Gen 12">
<meta property="og:image"
      content=".../prodotti/1-thinkpad-x1-carbon-gen-12/opengraph-image">
<meta property="og:image:width" content="1200">
<meta property="og:image:height" content="630">
<meta name="twitter:card" content="summary_large_image">
```

- Nessuno di questi tag è scritto a mano: li produce `generateMetadata()` partendo da `name`, `brand`, `category` e `description`.
- `title.template` nel layout fa il resto: la pagina dichiara solo “ThinkPad X1 Carbon Gen 12”.
- `metadataBase` trasforma i percorsi relativi in indirizzi assoluti: un `canonical` relativo non vuol dire niente per un crawler.

JSON-LD: il cuore della SEO di un negozio

```
{ "@context": "https://schema.org", "@type": "Product",
  "name": "ThinkPad X1 Carbon Gen 12", "sku": "MC-1",
  "category": "Portatili",
  "description": "Ultrabook 14\" con Intel Core Ultra 7, 32GB RAM, 1TB SSD...",
  "url": "http://localhost:3000/prodotti/1-thinkpad-x1-carbon-gen-12",
  "brand": { "@type": "Brand", "name": "Lenovo" },
  "offers": { "@type": "Offer",
    "price": "1899.00", "priceCurrency": "EUR",
    "itemCondition": "https://schema.org/NewCondition",
    "availability": "https://schema.org/InStock",
    "seller": { "@type": "Organization", "name": "minicommerce" } } }
```

- `availability`: `InStock` se `stock > 0`, altrimenti `OutOfStock`. Lo stesso dato del badge "Esaurito" del catalogo.
- `aggregateRating` darebbe le stelline nei risultati, ma **le recensioni non esistono** nel modello dati: aggiungerle è un esercizio, non si inventano.

Il JSON-LD generato dai dati veri

```
// web/lib/seo.ts
export function productJsonLd(prodotto: Product) {
  return {
    '@context': 'https://schema.org',
    '@type': 'Product',
    name: prodotto.name,
    sku: `MC-${prodotto.id}`,
    category: prodotto.category,
    url: absoluteUrl(productPath(prodotto)),
    ...(prodotto.brand ? { brand: { '@type': 'Brand', name: prodotto.brand } } : {}),
    offers: {
      '@type': 'Offer',
      price: prodotto.price.toFixed(2), priceCurrency: 'EUR',
      availability: prodotto.stock > 0
        ? 'https://schema.org/InStock'
        : 'https://schema.org/OutOfStock',
    },
  }
}
```

Quattro blocchi per pagina: `OnlineStore` e `WebSite` nel layout, `BreadcrumbList` e `Product` nella scheda. Da validare con il Rich Results Test di Google.

Il JSON-LD si stampa, non si costruisce a mano

```tsx
// web/components/StructuredData.tsx
export function StructuredData({ data }: { data: Record<string, unknown> }) {
  return (
    <script
      type="application/ld+json"
      dangerouslySetInnerHTML={{
        __html: JSON.stringify(data).replace(/</g, '\\u003c'),
      }}
    />
  )
}
```

- `dangerouslySetInnerHTML` qui è obbligato: React non sa mettere testo dentro uno `<script>`.
- Il rischio è reale, quindi lo chiudiamo a mano: se una descrizione contenesse `</script>` il browser chiuderebbe il blocco e il resto diventerebbe HTML eseguibile. Sostituendo `<` con `\u003c` il JSON resta valido e il tag non si può chiudere per sbaglio.
- È la stessa classe di problemi dell'XSS del modulo 6, in un posto dove nessuno la cerca.

Le immagini del catalogo: il nodo del base64

- Nel seed `imageUrl` è sempre `null`: il caso "nessuna immagine" non è un dettaglio, è il caso normale, e va disegnato bene.
- Quando l'admin carica una foto, il form la legge con `FileReader.readAsDataURL`: finisce nel database come **data URL base64**, fino a 5 MB.
- `next/image` non può ottimizzare un data URL: lo riconosce e imposta `unoptimized`, quindi lo lascia passare *così com'è*. Niente errori, ma niente WebP, niente ridimensionamento e 5 MB dentro l'HTML.
- Per le anteprime social ce la caviamo: l'`og:image` non è la foto dell'articolo, è un'immagine generata da noi (fra poche slide).
- **La soluzione vera** è l'upload su disco o su storage, con l'API che restituisce un URL. È l'esercizio che collega questo modulo al modulo 3.

Core Web Vitals su una pagina prodotto

Metrica	Misura	Buono	Cosa la rovina qui
LCP	quando appare l'elemento più grande	$< 2{,}5$ s	la foto dell'articolo: fino a 5 MB di base64 non ottimizzabili
INP	reattività ai click/tap	< 200 ms	idratare tutta la pagina invece della sola isola
CLS	quanto "salta" il layout	$< 0{,}1$	immagine senza `width`/`height`, prezzo che arriva dopo

- Sono un **fattore di ranking** e, soprattutto, misurano ciò che l'utente sente.
- Strumenti: Lighthouse (laboratorio), PageSpeed Insights e Search Console (utenti Chrome reali).
- Una SPA fatica su LCP; una pagina prodotto statica passa senza sforzo.

Next.js in due minuti

- Il framework React "full-stack" più diffuso: routing su file system, SSR/SSG/ISR, immagini ottimizzate, API route.
- **App Router**: **React Server Components** per default. I componenti girano sul server e mandano HTML; solo quelli marcati `'use client'` arrivano al browser.
- Sviluppato da Vercel; si distribuisce su Vercel con zero configurazione, o come server Node/Docker ovunque.
- Noi usiamo la versione **16**, quella che esce oggi da `create-next-app`: cache espliciti e guscio statico spedito subito.
- Per la vetrina di minicommerce è *molto*: lo usiamo perché conoscete React e perché lo troverete nelle aziende.

Il comando, e la struttura che ne esce

```
npx create-next-app@latest web --ts --app --tailwind \
    --no-src-dir --import-alias "@/*" --use-npm
```

```
web/
  app/
    layout.tsx                 intestazione, pie' di pagina, metadati comuni
    page.tsx                   /                 CATALOGO: dipende dai filtri
    not-found.tsx              la pagina 404
    robots.ts  sitemap.ts      generano /robots.txt e /sitemap.xml
    chi-siamo/page.tsx         pagina statica pura, nessun dato
    categorie/page.tsx         /categorie
    categorie/[slug]/page.tsx  /categorie/monitor
    prodotti/[slug]/page.tsx   /prodotti/1-thinkpad-x1-carbon-gen-12
    prodotti/[slug]/opengraph-image.tsx   l'anteprima social, una per articolo
    api/revalidate/route.ts    butta via la cache quando i dati cambiano
  components/                  Header, Footer, ProductCard, FilterForm, ...
  lib/api.ts                   il client dell'API, versione server
  lib/seo.ts  lib/slug.ts  lib/format.ts
```

Ogni cartella con `page.tsx` è una route; `layout.tsx` avvolge le figlie; `[slug]` è un parametro.

Server Component: dati senza useEffect

```tsx
// web/app/categorie/[slug]/page.tsx  -- gira SOLO sul server
export default async function CategoryPage({
  params,
}: PageProps<'/categorie/[slug]'>) {
  const { slug } = await params;
  const categoria = await trovaCategoria(slug);
  if (!categoria) notFound();

  const prodotti = await getProductsByCategory(categoria.name);

  return (
    <main>
      <h1>{categoria.name}</h1>
      <p>{categoria.description}</p>
      <ProductGrid prodotti={prodotti} />
    </main>
  );
}
```

In Next 16 `params` è una `Promise`: si aspetta con `await`. Il tipo `PageProps<'/categorie/[slug]'>` lo genera Next dalle cartelle che esistono davvero.

Cosa è cambiato rispetto al catalogo SPA

- Niente `useState`, niente `useEffect`, niente spinner: è una funzione `async` che restituisce JSX.
- La chiamata a `GET /api/products?category=Monitor` avviene **sul server**, al build: l'utente non la aspetta.
- Il client riceve HTML finito e **zero JavaScript** per questo componente.
- L'API non ha bisogno di CORS per queste chiamate: partono da Node, non dal browser.
- Niente `axios` e niente token: `lib/api.ts` usa `fetch` e legge solo rotte pubbliche. L'indirizzo sta in `API_URL`, *senza* prefisso pubblico, quindi resta sul server.

Una pagina HTML per articolo: generateStaticParams

```tsx
// web/app/prodotti/[slug]/page.tsx

/** L'elenco delle pagine da pregenerare: una per articolo. */
export async function generateStaticParams() {
  const prodotti = await getCatalog();                    // GET /api/products
  return prodotti.map((p) => ({ slug: productSlug(p) }));
}

export default async function ProductPage({
  params,
}: PageProps<'/prodotti/[slug]'>) {
  const { slug } = await params;
  const id = idFromSlug(slug);                // "1-thinkpad-..." -> 1
  if (id === null) notFound();

  const prodotto = await getProduct(id);
  if (!prodotto) notFound();

  // Slug vecchio (nome cambiato)? 308 verso l'indirizzo canonico.
  const slugCanonico = productSlug(prodotto);
  if (slug !== slugCanonico) permanentRedirect(`/prodotti/${slugCanonico}`);
```

`generateStaticParams` gira durante `next build`: se il backend non è avviato, la build si ferma. È il patto del rendering statico, i dati devono esistere prima dei visitatori.

Lo slug: perché l'id sta davanti

- `/prodotti/1` non dice niente a chi legge il link, né a un motore di ricerca. `/prodotti/1-thinkpad-x1-carbon-gen-12` sì.
- Il nome di un articolo **cambia**; l'id no. Con l'id in testa la pagina si ritrova sempre, anche dopo una rinomina.
- Se lo slug non è quello canonico, la pagina risponde **308** verso quello giusto: l'utente arriva dove voleva e Google aggiorna il suo indice.
- Lo slug non esiste nel database: lo calcoliamo dal nome con `slugify()` (`normalize('NFD')` per togliere gli accenti, tutto il resto diventa un trattino).
- Aggiungere una colonna `slug` al database, per poterlo scegliere a mano, è un esercizio.

Next 16: cosa è in cache lo decidete voi

```ts
// web/lib/api.ts
import { cacheLife, cacheTag } from 'next/cache'

/** Catalogo completo: alimenta pagine statiche, sitemap e staticParams. */
export async function getCatalog(): Promise<Product[]> {
  'use cache'
  cacheTag('prodotti')
  cacheLife('hours')
  return chiamaApi<Product[]>('/products?sort=name&order=asc')
}

/** La ricerca con i filtri NON e' in cache: cambia a ogni visita. */
export async function searchProducts(filtri: ProductFilters) {
  const query = new URLSearchParams(/* ... */)
  return chiamaApi<Product[]>(`/products?${query}`)   // cache: 'no-store'
}
```

- Con `cacheComponents` acceso la regola è una: **niente è in cache se non lo chiedete voi**.
- `'use cache'` rende la funzione memorizzabile, `cacheTag` le dà un'etichetta per invalidarla, `cacheLife` dice quanto può restare.
- "Statico" e "dinamico" diventano una scelta scritta nel codice, invece di un effetto collaterale di quale funzione si è chiamata.

L'elenco di `next build`: il riassunto più onesto

Route	Modalità	
`/`	PPR	guscio statico, risultati in streaming
`/categorie`	statica	
`/categorie/[slug]`	SSG $\times$ 7	una per categoria
`/chi-siamo`	statica	nessun dato
`/prodotti/[slug]`	SSG $\times$ 16	una per articolo
`/prodotti/[slug]/opengraph-image`	SSG $\times$ 16	anche le immagini
`/robots.txt`, `/sitemap.xml`	statiche	
`/api/revalidate`	dinamica	è un'azione

- Nell'output di `next build` le trovate marcate con un cerchio vuoto (statica), pieno (SSG), mezzo pieno (PPR) e una *f* corsiva (dinamica).
- 16 articoli, 16 pagine, 16 immagini di anteprima: tutto pronto **prima** del primo visitatore. Tempo totale della generazione: circa due secondi.
- Il catalogo con i filtri è l'unica pagina che non si può pregenerare, e si vede.

I filtri senza una riga di JavaScript

```
<form method="get" action="/">
  <input name="search" type="search" value="cuffie">
  <select name="category">...</select>
  <input name="minPrice" type="number"> <input name="maxPrice" type="number">
  <select name="sort">...</select> <select name="order">...</select>
  <button type="submit">Applica i filtri</button>
</form>
```

- Nella SPA questo pannello è uno stato React tenuto in sincronia con `useSearchParams`. Qui è un `<form method="get">`: il browser compone la query string da solo e il server risponde con l'HTML giusto.
- Funziona con JavaScript disattivato, funziona dentro un crawler, e il risultato si può copiare e incollare come link.
- Il prezzo da pagare: i campi vuoti finiscono comunque nell'URL (`?search=&brand=`). Il backend li ignora, ma per un motore di ricerca sono indirizzi diversi con lo stesso contenuto.

La pagina filtrata non va indicizzata

```tsx
// web/app/page.tsx
export async function generateMetadata({ searchParams }: PageProps<'/'>) {
  const filtri = leggiFiltri(await searchParams)

  if (contaFiltri(filtri) === 0) {
    return { title: { absolute: 'minicommerce - ...' },
             alternates: { canonical: '/' } }
  }
  return {
    title: `Ricerca: ${pezzi.join(', ')}`,
    alternates: { canonical: '/' },
    robots: { index: false, follow: true },   // <-- noindex, follow
  }
}
```

- `?search=mouse&minPrice=10&order=desc` e le sue infinite varianti mostrano pezzi della stessa lista. Lasciarle indicizzare significa far spendere al crawler il suo budget su migliaia di doppioni invece che sulle schede prodotto.
- `follow` resta attivo: i link dentro la pagina vanno seguiti, la pagina no.
- `canonical` punta al catalogo pulito: a Google diciamo qual è l'indirizzo che conta.

“E quando cambia un prezzo?”

```
// web/app/api/revalidate/route.ts
export async function POST(richiesta: NextRequest) {
  const segreto = process.env.REVALIDATE_SECRET
  const fornito = richiesta.headers.get('x-revalidate-secret')
  if (fornito !== segreto) {
    return Response.json({ error: 'Segreto mancante o sbagliato' },
                         { status: 401 })
  }
  const { tag } = await richiesta.json()        // 'prodotti', 'prodotto-7', ...
  revalidateTag(tag, { expire: 0 })             // butta via la copia in cache
  return Response.json({ ok: true, tag })
}
```

- È la domanda che arriva sempre, ed è quella giusta. Risposta: si butta via la copia in cache, e la prima visita successiva rigenera l’HTML. **Nessuna build, nessun deploy**.
- Il secondo argomento dice quanto si può ancora servire la copia vecchia: con `'max'` l’aggiornamento avviene di nascosto, con `{ expire: 0 }` la copia vecchia va subito in pattumiera. In produzione si preferisce il primo; a lezione serve vedere il cambiamento subito.
- Serve un segreto in variabile d’ambiente: un indirizzo pubblico che costa lavoro al server è un invito a farsi male.

La prova del nove, in tre comandi

```
# 1. l'admin cambia il prezzo (dalla SPA, oppure con l'API)
#    prezzo nel database: 1555      pagina: ancora 1899,00 EUR  (in cache)

# 2. si butta via la copia in cache
curl -X POST "http://localhost:3000/api/revalidate" ‘
     -H "x-revalidate-secret: cambiami" ‘
     -H "content-type: application/json" ‘
     -d '{"tag":"prodotti"}'
# -> {"ok":true,"tag":"prodotti"}

# 3. si ricarica la pagina:  1.555,00 EUR
```

- Senza il segreto: `401 Segreto mancante o sbagliato`.
- In fondo alla scheda c'è scritto "Dati in cache dal …": quella data è il momento in cui l'HTML è stato generato, non quello della visita. Ricaricate dieci volte e non cambia; invalidate l'etichetta e cambia.
- **L'esercizio naturale**: chiamare questa rotta dal backend dentro `PUT /api/products/:id`, così il sito si aggiorna da solo appena si salva un articolo.

404 e 308: lo stato HTTP conta

```
curl -I http://localhost:3000/prodotti/999-non-esiste    # 404
curl -I http://localhost:3000/prodotti/1-nome-sbagliato  # 308 -> slug giusto
curl -I http://localhost:5173/pagina/che/non/esiste      # 200 (!) nella SPA
```

- Un motore di ricerca che riceve 404 toglie l'indirizzo dall'indice. Nella SPA *qualunque* indirizzo sbagliato risponde 200 con la pagina vuota: per un crawler sono tutte pagine valide e identiche.
- Una finezza da provare: alla **primissima** richiesta di un indirizzo mai visto, Next spedisce subito il guscio statico (stato 200) e completa la pagina in streaming; dalla seconda in poi l'esito è in cache e lo stato è quello giusto. Lanciate il comando due volte e si vede.
- Nel frattempo la pagina dichiara comunque `noindex`, quindi nell'indice non ci finisce.

L'anteprima social, una per articolo

```tsx
// web/app/prodotti/[slug]/opengraph-image.tsx
export const size = { width: 1200, height: 630 }
export const contentType = 'image/png'

export default async function OpenGraphImage({ params }) {
  const prodotto = await getProduct(idFromSlug((await params).slug))
  return new ImageResponse(
    <div style={{ display: 'flex', flexDirection: 'column', padding: 72,
                  backgroundColor: '#0a0a0a', color: '#fafafa' }}>
      <div style={{ fontSize: 68, fontWeight: 700 }}>{prodotto.name}</div>
      <div style={{ fontSize: 56 }}>{formatPrice(prodotto.price)}</div>
    </div>,
    size,
  )
}
```

- Il file si chiama `opengraph-image` e sta nella cartella della pagina: Next aggiunge da solo i `<meta property="og:image">` con indirizzo e dimensioni.
- Dentro `ImageResponse` non c'è un browser: c'è **satori**, che sa disegnare flexbox e testo e nient'altro. Stili inline, niente Tailwind.
- Sono 16 PNG generati al build. È anche il modo di avere un'anteprima decente *senza* avere le foto degli articoli.

sitemap.xml e robots.txt nascono dai dati

```ts
// web/app/sitemap.ts
export default async function sitemap(): Promise<MetadataRoute.Sitemap> {
  const [prodotti, categorie] = await Promise.all([getCatalog(), getCategories()])
  return [
    { url: absoluteUrl('/'), changeFrequency: 'daily', priority: 1 },
    ...categorie.map((c) => ({ url: absoluteUrl(`/categorie/${categorySlug(c)}`) })),
    ...prodotti.map((p) => ({ url: absoluteUrl(`/prodotti/${productSlug(p)}`),
                              lastModified: new Date(p.createdAt) })),
  ]
}
```

- 26 indirizzi: tre pagine fisse, sette categorie, sedici articoli. Nessuno li ha scritti a mano, e un articolo nuovo ci finisce dentro alla prima rigenerazione.
- `lastModified` non è decorativo: dice al crawler cosa vale la pena riscaricare. Non metteteci "adesso", o l'informazione non vale più niente.
- `robots.ts` dichiara dov'è la sitemap e cosa non vale la pena leggere (`/api/`). Attenzione all'equivoco: `disallow` non nasconde e non protegge niente.

Due cose che la vetrina *non* usa

Nessun `'use client'`

- Nella vetrina non c'è un solo componente client: i filtri sono un `<form>`, il tema è CSS, il carrello sta nella SPA.
- `'use client'` segna il confine: da lì in giù il codice va anche al browser.
- Serve quando c'è davvero interazione locale. Qui non c'era.

Nessuna Server Action

- `'use server'` esegue sul server la funzione che gestisce un form, senza scrivere una API.
- La vetrina non ha form da inviare: l'unica scrittura è `POST /api/revalidate`, che è una rotta.
- Nel repository però c'è un progetto intero costruito su di esse, ed è la prossima slide.

Il tema scuro, per dire

Niente `next-themes` e nessuna classe sul `<html>`: solo `prefers-color-scheme`. Nessun JavaScript, nessuna idratazione, nessun lampeggio.

La stessa applicazione in un progetto solo: `singleton/`

Sul ramo `web/nextjs` c'è una seconda risposta, in una cartella a parte: non la vetrina *accanto* alla SPA, ma **tutto il negozio dentro Next**. Catalogo, login, carrello e admin, senza API REST in mezzo.

	SPA + Express (modulo 1)	`singleton/`
Lettura del catalogo	il browser chiama `GET /api/products`	un Server Component chiama `listProducts()`
Scrittura	`POST /api/products` con token Bearer	una Server Action scrive su SQLite
Sessione	JWT in `localStorage`	JWT in un cookie `httpOnly`
Processi da avviare	2 (Vite e Express)	1

L'API non scompare: cambia natura

Le stesse quattro operazioni ci sono tutte, ma da *contratto HTTP* diventano **chiamate di funzione tipizzate**. Il confine fra client e server resta, e resta sorvegliato: cambia solo che non è più una rete.

singleton/: leggere e scrivere senza una rotta

```ts
// singleton/app/page.tsx -- il catalogo, filtri compresi, e' un Server Component
export default async function CatalogPage({ searchParams }: PageProps<'/'>) {
  const params = await searchParams          // i filtri stanno nella query string
  const session = await getSession()         // dal cookie firmato, non da un header
  const products = listProducts({ search, category, sort, order })   // SQLite, qui
  return <div>{products.map((p) => <ProductCard key={p.id} product={p} />)}</div>
}
```

```ts
// singleton/server/actions/cart.ts -- la scrittura, senza una rotta
'use server'
export async function addToCartAction(productId: number, quantity = 1) {
  const session = await requireCustomer()    // non c'e' un 401: c'e' un'eccezione
  db.insert(cartItems).values({ userId: session.userId, productId, quantity }).run()
  revalidatePath('/carrello')                // la pagina si rifa' da sola
}
```

- `lib/db/index.ts` è marcato `server-only`: se un componente client provasse a importarlo, la build *fallisce* invece di spedire il driver SQLite al browser.
- `serverExternalPackages: ['better-sqlite3']` in `next.config.ts`: è un modulo nativo, deve restare un `require()` di Node.

Tutto in uno: cosa si guadagna, cosa si perde

+ Si guadagna

- Un processo invece di due, un deploy invece di due.
- I tipi e la validazione si scrivono **una volta**.
- Niente token in `localStorage`, niente CORS, niente proxy di Vite.

– Si perde

- **L'API pubblica**: Electron, Capacitor e le app native dei moduli 2, 3 e 5 non hanno più niente da chiamare.
- Il server diventa obbligatorio: serve un hosting con Node.
- Il contratto (`docs/API.md`) non ha più un posto dove vivere.

Perché nel corso restano entrambi

Rifare l'applicazione senza API è costato circa **3.400 righe**, e il backend Express non si è toccato: ci parlano altri quattro client. Con `DATABASE_URL=../backend/minicommerce.db` le due versioni lavorano sugli **stessi dati**: l'unico stato vero è il database.

Astro: nato per i siti di contenuto

- Genera **HTML statico con zero JavaScript** per default. Ogni componente interattivo è un'**isola** che dichiarate esplicitamente.
- Le isole possono essere in **React**, Vue, Svelte, Solid: riusate i componenti che sapete già scrivere.
- **Content collections**: Markdown/MDX con schema validato (zod), perfetto per tutto ciò che scrive una persona.
- `astro build` produce una cartella di file: hosting statico ovunque, anche dove Node non esiste. Punteggi Lighthouse da 100 senza sforzo.
- Nel 2026 è la scelta più comune per landing, blog, documentazione, vetrine di contenuto.

Si può usare Astro per minicommerce?

Pezzo del progetto	Astro è adatto?
Guide, recensioni, pagine informative	+ **Sì**: è esattamente il suo mestiere
Catalogo con filtri e ricerca	– No: dipende dalla query string, cambia a ogni richiesta
Scheda prodotto con prezzo e stock	+ Sa pregenerarla, – ma non ha una risposta pronta a "e quando cambia un prezzo?"
Carrello, login, area admin	– No: hanno utente, token e stato

La domanda giusta non è "si può", è "su che pezzo"

Astro genera le pagine in anticipo: va a nozze con ciò che cambia *quando lo decide chi scrive*, va male con ciò che cambia da solo o che dipende da chi guarda. Per le schede prodotto la risposta è Next con `revalidateTag`; per il blog è Astro, e non c'è gara.

Il comando, e la struttura che ne esce

```
npm create astro@latest blog -- --template minimal
cd blog && npx astro add react sitemap tailwind
npm install @astrojs/rss
```

```
blog/
  astro.config.mjs           site, output: 'static', le tre integrazioni
  src/
    content.config.ts        la forma del frontmatter: un errore ferma la build
    content/blog/*.md        i sei articoli
    layouts/BaseLayout.astro head, metadati, Open Graph, JSON-LD
    components/
      PrezzoLive.tsx         LA sola isola React del sito
      ProductCard.astro      la scheda di un articolo del catalogo
    lib/shop.ts              legge il catalogo dall'API, solo in fase di build
    pages/
      index.astro            l'elenco degli articoli
      blog/[...slug].astro   una pagina per post (getStaticPaths)
      tag/[tag].astro        una pagina per argomento
      rss.xml.ts  robots.txt.ts  404.astro
```

Template `minimal`: partiamo da una pagina vuota, così si vede nascere ogni pezzo. Gli altri template arrivano con un blog già fatto, che a lezione è come leggere la soluzione prima dell'esercizio.

I contenuti hanno uno schema

```
// blog/src/content.config.ts
const blog = defineCollection({
  loader: glob({ pattern: '**/*.md', base: './src/content/blog' }),
  schema: z.object({
    title: z.string().min(10).max(70),
    description: z.string().min(50).max(160),
    pubDate: z.coerce.date(),
    author: z.string().default('Redazione minicommerce'),
    tags: z.array(z.string()).min(1),
    /** Gli id degli articoli del catalogo di cui parla il post. */
    prodotti: z.array(z.number().int().positive()).default([]),
    bozza: z.boolean().default(false),
  }),
})
export const collections = { blog }
```

- Da qui nascono **i tipi**: nelle pagine `post.data.pubDate` è una `Date` vera, con l'autocompletamento.
- I limiti sulla `description` non sono pignoleria: sotto i 50 caratteri non dice niente, sopra i 160 Google la taglia. La regola SEO diventa un controllo, una volta per tutte.

Lo schema ferma la build, non il visitatore

```
# scrivete "pubdate:" invece di "pubDate:", oppure accorciate la description
npm run build

[InvalidContentEntryDataError] blog -> portatile-per-studiare-informatica
  data does not match collection schema.
  description: Too small: expected string to have >=50 characters
```

- Trenta secondi di dimostrazione, e si capisce a cosa serve un file di configurazione dei contenuti.
- Un contenuto sbagliato non arriva in produzione: si ferma sul computer di chi scrive, con il nome del file e il nome del campo.
- È la stessa idea della validazione zod sul form del modulo 1, spostata un passo più indietro: non “l’utente ha sbagliato”, ma “l’autore ha sbagliato”.

Una pagina per post

```
---
// blog/src/pages/blog/[...slug].astro  -- questa parte gira al BUILD
export async function getStaticPaths() {
  const post = await getCollection('blog', ({ data }) => !data.bozza);
  return post.map((articolo) => ({
    params: { slug: articolo.id },
    props: { articolo },
  }));
}

const { articolo } = Astro.props;
const { Content } = await render(articolo);              // Markdown -> HTML
const prodotti = await getProducts(articolo.data.prodotti);  // dall'API
---
<BaseLayout title={articolo.data.title} description={articolo.data.description}>
  <h1>{articolo.data.title}</h1>
  <div class="prosa"><Content /></div>
  {prodotti.map((prodotto) => <ProductCard prodotto={prodotto} />)}
</BaseLayout>
```

`getStaticPaths` è il cuore di un generatore statico: dice "ecco l'elenco delle pagine che devono esistere". Astro le costruisce tutte e il risultato è una cartella di file `.html`.

Il filo con il negozio, e il suo difetto

- Ogni post dichiara nel frontmatter gli `prodotti` di cui parla. `lib/shop.ts` li legge dall'API **una volta per build** (sei post, due articoli ciascuno, una sola chiamata).
- Quindi il prezzo scritto in pagina è il prezzo **del giorno della pubblicazione**. Se domani cambia nel catalogo, qui resta quello vecchio finché il blog non viene rigenerato.
- Il difetto è strutturale: invece di nasconderlo, lo raccontiamo ai lettori in un post e lo correggiamo dove serve.
- `lib/shop.ts` **non** fa fallire la build se l'API è spenta: qui il contenuto sono gli articoli, i prezzi sono un extra. Nella vetrina Next la scelta è opposta, e per il motivo opposto: là il contenuto è il prodotto.

L'isola: rendere dinamico il numero, non il sito

```
// blog/src/components/PrezzoLive.tsx     (React, idratato nel browser)
export default function PrezzoLive({ productId, prezzoAlBuild }) {
  const [prezzo, setPrezzo] = useState(prezzoAlBuild)
  const [stato, setStato] = useState('attesa')

  useEffect(() => {
    const controllo = new AbortController()
    fetch(`${API_URL}/products/${productId}`, { signal: controllo.signal })
      .then((r) => r.json())
      .then((p) => { setPrezzo(p.price)
                     setStato(p.price === prezzoAlBuild ? 'confermato' : 'cambiato') })
      .catch(() => { if (!controllo.signal.aborted) setStato('errore') })
    return () => controllo.abort()
  }, [productId, prezzoAlBuild])
```

```
<PrezzoLive productId={prodotto.id} prezzoAlBuild={prodotto.price} client:visible />
```

Stesso schema dell'hook `useProducts` del modulo 1, `AbortController` compreso. Invece di rendere dinamico tutto il sito per un numero, rendiamo dinamico **il numero**.

client:visible, e il costo del JavaScript

Quanto JavaScript scarica un lettore

- Home ed elenchi per tag: **un solo** `<script>`, e contiene il JSON-LD. Zero JavaScript eseguibile.
- Pagina di un post: tre script, cioè il runtime delle isole e il componente del prezzo, *quando* serve.
- Si paga solo quello che si usa.

Le direttive

- `client:load`: subito.
- `client:idle`: quando il browser è libero.
- `client:visible`: quando entra nello schermo (la nostra).
- `client:only`: mai sul server.
- Nessuna direttiva: HTML puro, zero JS.

La prova da fare in aula

Cambiate un prezzo dal negozio e ricaricate il post *senza* rifare la build: sotto il prezzo compare "prezzo aggiornato adesso", con il vecchio sbarrato. Poi disattivate JavaScript: resta il prezzo pubblicato, con scritto che è quello. L'isola **aggiunge**, non sostituisce.

Feed, sitemap e robots: file che non si scrivono

```ts
// blog/src/pages/rss.xml.ts
export async function GET(context: APIContext) {
  const post = await getCollection('blog', ({ data }) => !data.bozza)
  return rss({
    title: 'Il blog di minicommerce',
    site: context.site!,            // da "site" in astro.config.mjs
    items: post.map((a) => ({
      title: a.data.title, description: a.data.description,
      pubDate: a.data.pubDate, link: `/blog/${a.id}/`,
      categories: a.data.tags,
    })),
    customData: '<language>it-it</language>',
  })
}
```

- Un file dentro `src/pages` che non finisce in `.astro` è una rotta che restituisce quello che vuole: XML qui, testo in `robots.txt.ts`.
- Sei post diventano **15 pagine** in un secondo e mezzo: sei articoli, sette archivi per tag, la home e la 404, più feed, robots e sitemap.
- Nessun elenco è scritto a mano: i tag nascono dai post, la sitemap dalle pagine costruite davvero.

E Astro, tutto in uno?

La domanda è legittima: se Next ha `singleton/`, perché il blog chiama ancora l'API?

- I pezzi ci sono tutti: Astro 7 ha le **azioni** (`astro:actions`) al posto delle rotte, la **sessione** in configurazione, le **isole server** (`server:defer`), e con `@astrojs/node` e `output: 'server'` diventa un processo come Next.
- Quindi sì, `singleton/` si potrebbe riscrivere in Astro: si otterrebbe una versione *meno comoda* di ciò che Next fa di mestiere, usando Astro contro la sua natura.
- Il "tutto in uno" che ha senso per il blog è un altro: `lib/shop.ts` legge SQLite con Drizzle *durante la build* invece di chiamare `/api/products`. Un file, e il backend non serve più per costruire il blog.

Il punto, e il suo prezzo

A un generatore statico la sorgente è indifferente: API, database, CMS, file. Conta **quando** legge, e legge al build. Ma il prezzo resta congelato come prima, e il blog si lega allo **schema del database** invece che al contratto HTTP: più comodo oggi, più fragile al primo `ALTER TABLE`.

Astro o Next? La tabella che serve davvero

Se dovete...	Scegliete
pubblicare contenuti scritti da persone	**Astro**
mandare al browser il meno JavaScript possibile	**Astro**
pubblicare su un hosting qualunque, senza Node	**Astro**
mostrare dati che cambiano da soli (prezzi, scorte)	**Next**
rigenerare una pagina alla volta quando un dato cambia	**Next**
gestire sessioni, form autenticati, azioni lato server	**Next**
fare tutto questo insieme	**tutti e due**, su pezzi diversi

- Entrambi sanno fare quasi tutto: la differenza è *cosa* rendono facile e cosa fanno pagare.
- Ed è esattamente ciò che fa minicommerce: quattro rami, quattro strumenti, un repository e un solo backend.

PHP nel 2026: il sito che il negozio ha già

- WordPress da solo serve oltre il 40% dei siti; PHP è il rendering lato server *originale*.
- Un negozio di informatica che vende online, oggi, molto probabilmente ha **WooCommerce** o **PrestaShop**: SSR puro, SEO già risolta, catalogo gestito dal cliente.
- **Laravel** è un framework moderno: routing, ORM, template Blade, code, autenticazione.
- Con **Inertia.js** Laravel serve pagine React: il server fa routing e dati, React disegna. Niente API REST da scrivere.
- Con **Livewire** si scrivono componenti interattivi in PHP, con pochissimo JS.

Laravel + Blade: la stessa scheda prodotto

```php
// routes/web.php
Route::get('/prodotti/{id}', function (int $id) {
    $articolo = Product::with('category')->findOrFail($id);
    return view('prodotti.show', ['articolo' => $articolo]);
});
```

```
{{-- resources/views/prodotti/show.blade.php --}}
@extends('layouts.app', ['title' => $articolo->name])
@section('content')
  <article>
    <h1>{{ $articolo->name }}</h1>
    <p>{{ $articolo->brand }} - {{ number_format($articolo->price, 2) }} EUR</p>
    <p>{{ $articolo->stock > 0 ? 'Disponibile' : 'Esaurito' }}</p>
  </article>
@endsection
```

- Ogni richiesta esegue PHP e produce HTML: SSR puro. Cache (Redis, CDN) per reggere il traffico.
- {{ }} fa l'escape: la protezione da XSS è il default, come in JSX.

CMS: quando il catalogo non è vostro

- Il cliente vuole cambiare testi, foto e promozioni da solo: un CMS è la risposta, non un file Markdown nel vostro repo.
- Due modi di usare WordPress:
  - **Classico**: tema PHP, plugin, hosting condiviso. Veloce da consegnare, lento e fragile nel tempo.
  - **Headless**: WordPress solo come backend (REST o GraphQL), il sito lo genera Astro/Next. Prestazioni del sito statico, editing familiare.
- Alternative headless nate per questo: **Strapi**, **Payload**, **Sanity**, **Directus**. Per i cataloghi: Shopify, Medusa, commercetools.
- Nel nostro caso il "CMS" è l'admin di minicommerce, e il webhook "articolo salvato" → `POST /api/revalidate` è lo stesso schema.

Il resto del panorama

- **Nuxt** (Vue), **SvelteKit** (Svelte), **React Router framework mode** (ex Remix): stessi concetti di Next con altri framework.
- **Hugo**, **Eleventy**: generatori statici senza framework JS. Velocissimi, ideali per documentazione.
- **HTMX**: HTML che chiede frammenti HTML al server. Con PHP, Go o Python permette interattività senza SPA.
- **Edge rendering**: Cloudflare Workers, Vercel Edge. SSR eseguito vicino all'utente.
- **Site builder** (Webflow, Framer, Shopify): per il cliente che non vuole un repo. Sappiate cosa sono e quando proporli.

Cosa c'è nei due rami

`web/nextjs` – la vetrina

- Cartella `web/`: 38 file aggiunti
- 16 schede prodotto + 7 categorie pregenerate
- sitemap, robots, 16 immagini di anteprima
- `POST /api/revalidate`
- Più `singleton/`: la stessa app senza API
- **Non tocca** né `frontend/` né `backend/`

`web/astro` – il blog

- Cartella `blog/`: 34 file aggiunti
- 6 post, 15 pagine, feed RSS
- Una sola isola React
- `lib/shop.ts` legge il catalogo al build
- Del negozio cambia **un solo file**: la voce "Blog" nell'header

Un repository, quattro rami

`master` + `web/nextjs` + `web/astro` + `desktop/electron` si fondono fra loro senza un conflitto: ogni ramo aggiunge una cartella e non tocca quelle degli altri. Gli unici due incompatibili sono Electron e Tauri *fra loro*, ed è giusto: sono due risposte alla stessa domanda.

I due rami di riferimento

```
git switch web/nextjs     # la vetrina in web/, e un secondo commit: singleton/
git switch web/astro      # il blog in blog/, e nient'altro di diverso

# oppure, da un clone pulito di master, la trasformazione completa:
node scripts/setup-nextjs.mjs    # create-next-app, i file, install, build
node scripts/setup-astro.mjs     # create astro, astro add, i file, build

git diff master..web/astro -- frontend/    # l'unico file toccato nel negozio
cd singleton && npm install && npm run db:push && npm run db:seed && npm run dev
                           # database suo, porta 3000: non pesta i piedi a nessuno
```

- Il messaggio di ogni commit di merge racconta tutto ciò che è stato fatto. Leggetelo, è la sintesi della lezione.
- I due script stampano ogni comando esterno prima di eseguirlo, e sono **idempotenti**: potete rilanciarli senza rompere niente.
- `setup-nextjs.mjs` controlla che l'API risponda prima di compilare: le pagine statiche nascono dai dati.
- Documentazione: `docs/WEB-NEXTJS.md` e `docs/WEB-ASTRO.md`, con gli esperimenti nell'ordine giusto e i problemi noti.

Laboratorio 4a

Laboratorio: La vetrina Next, e poi misurarla

1. Da un clone pulito: `node scripts/setup-nextjs.mjs` con il backend avviato. Leggere l'elenco di `next build` e dire, riga per riga, perché quella pagina è statica o dinamica.
2. `node scripts/confronto-seo.mjs`: confrontare SPA e vetrina sullo stesso articolo. Poi Lighthouse su entrambe, da telefono.
3. Cambiare un prezzo dall'admin, verificare che la vetrina mostri ancora quello vecchio, invalidare con `POST /api/revalidate` e ricaricare.
4. **Chiudere il cerchio**: far chiamare la rotta di invalidazione dal backend dentro `PUT /api/products/:id`, con il segreto in `.env`.
5. Aggiungere la **paginazione** al catalogo e una pagina `/cerca` con il suo `noindex`.
6. Validare il JSON-LD di una scheda col Rich Results Test di Google.
7. Avviare `singleton/`: dov'è finita la validazione dell'API, dov'è la sessione, e cosa dovrebbe cambiare nell'Electron del modulo 2 per parlare con *questa* versione?

Laboratorio 4b

Laboratorio: Il blog Astro, e il confine fra statico e dinamico

1. `node scripts/setup-astro.mjs`, poi `npm run build`: servire `dist/` senza Node (`npx serve dist`).
2. Scrivere un settimo post che cita due articoli veri. Prima rompere il frontmatter di proposito, per vedere la build fermarsi.
3. Cambiare un prezzo dal negozio e ricaricare il post *senza* rifare la build: osservare l'isola. Poi disattivare JavaScript.
4. Contare gli `<script>` nella home del blog e in una pagina della SPA, e spiegare la differenza.
5. Una copertina ai post con `astro:assets`.
6. **Togliere l'API al blog**: far leggere a `lib/shop.ts` il database con Drizzle durante la build. Poi dieci righe: quale strumento per quale pezzo di questo negozio, e quale delle due versioni si rompe prima?

Domande di verifica

Domande di verifica

- Cos'è l'idratazione, e perché un Server Component non può usare `useState`?
- Cosa cambia fra `client:load` e `client:visible`?
- Cosa fa `'use cache'`, e cosa fa `revalidateTag`?
- Perché una pagina filtrata dichiara `noindex` ma non `nofollow`?
- Come si deduce `offers.availability` dai dati di minicommerce?
- Perché il carrello resta nella SPA e non entra nella vetrina?
- Perché un'immagine base64 non viene ottimizzata da `next/image`?
- Perché il prezzo in un post del blog può essere vecchio, e cosa lo corregge?
- `singleton/` non ha API REST: chi, nel repository, se ne accorgerebbe per primo?